

Working with remote repositories on GitHub

A practical guide to local and remote repositories, synchronization, collaboration, and documentation with Git and GitHub.



Local vs. Remote Repository

The most important basic concept when working with Git and GitHub: there are two places where your project lives.

Local = Workstation

The repository lives on your computer. This is where you edit files, make commits, and experiment in peace.

Remote = Central Storage

The repository lives on a server, e.g., GitHub. This is where your project is stored online, shared, and synchronized.

Git = Version Control

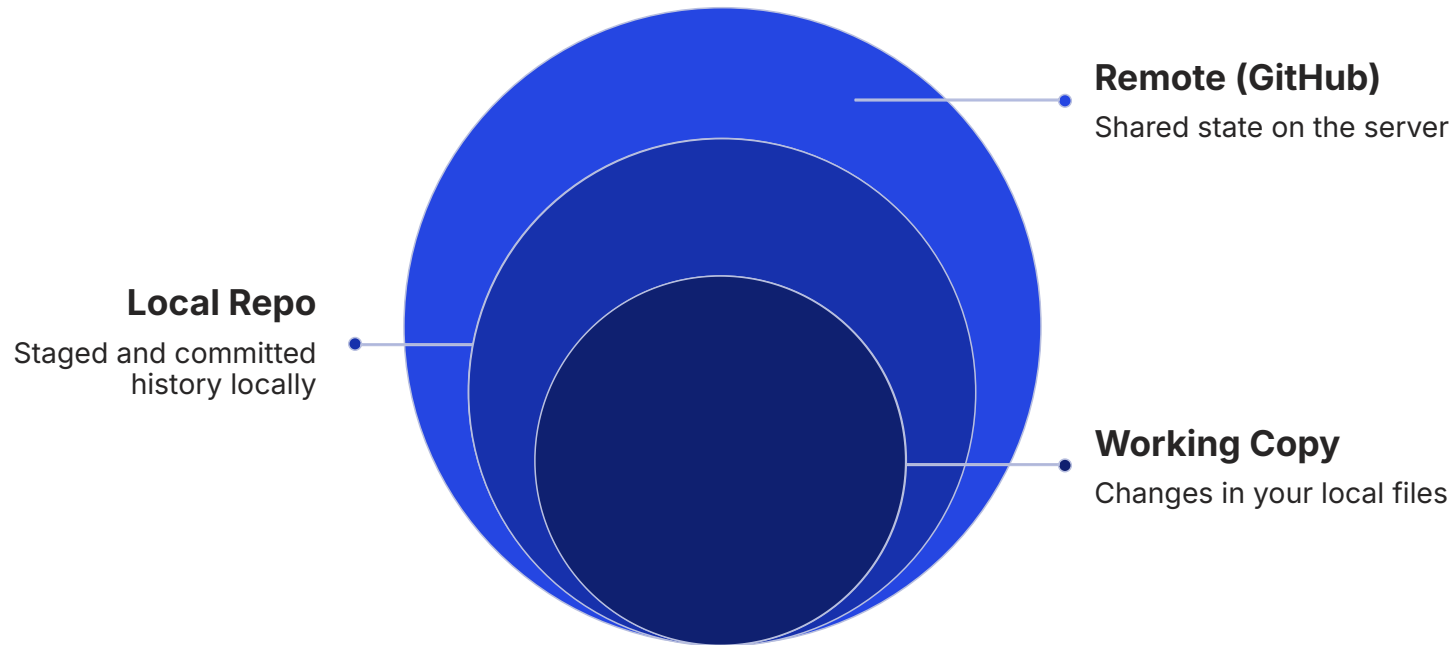
The system that cleanly tracks changes and coordinates local and remote states.

GitHub = Platform

The platform where you manage Git repositories online, share them, and collaborate with others.

Three States – Keeping Them Separate

Many Git problems arise not because Git is "complicated", but because these three states are not clearly distinguished:



If you have these three levels clear in your head, you will immediately understand most Git error messages better.

Create a GitHub account

To use GitHub, you need a personal account. You can register directly at **github.com**.

01

Visit github.com & click Sign up

02

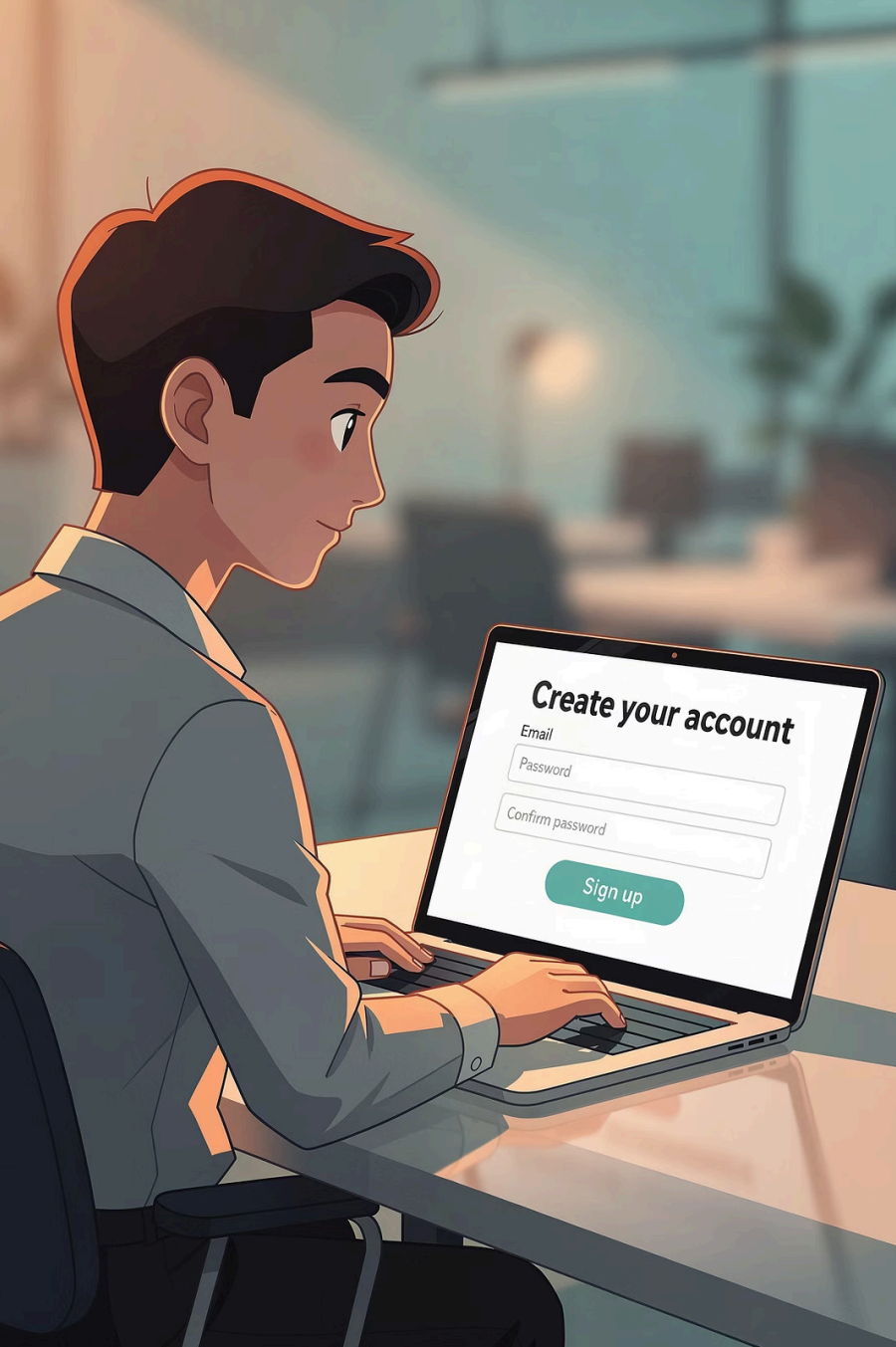
Choose a username, email, and password

03

Verify your email address

04

Create repositories, publish projects, and collaborate



What you can do with your GitHub account



Manage repositories

Create your own repositories, manage them as **public** or **private**, and store existing projects online.



Collaborate

Develop with others, create pull requests, and manage issues.



Document

Display project documentation via README files directly in the repository.

❏ **Important:** For Git operations via HTTPS, your GitHub password is not sufficient. GitHub requires secure authentication via **Personal Access Token**, browser sign-in, or **SSH key**.

HTTPS or SSH?

There are two common ways to connect to GitHub. Both are officially documented and supported by GitHub.

HTTPS

Simple for getting started. Authentication via token or login dialog. Recommended for beginners.

SSH

Very convenient in the long run. Set up an SSH key once – afterwards often more comfortable. Recommended for regular use.

If you are just starting out, **HTTPS** is often easier to understand. If you work with GitHub regularly, **SSH** is often more comfortable in the long term.



CONNECTION

Connecting a local repository to GitHub

Your local repository only knows about GitHub once you tell it: "This online repository there is my remote." For this, the name **origin** is usually used – a common standard name, not a magic word.

Typical Connection Workflow

01

Initialize locally

`git init` → `git add .` → `git commit -m "First commit"`

03

Connect remote

`git remote add origin https://github.com/YOUR-NAME/my-project.git`

You can use `git remote -v` at any time to verify that the connection has been set up correctly.

02

Create empty repository on GitHub

Create a new, empty repository on github.com (e.g., `my-project`).

04

Perform first push

`git branch -M main` → `git push -u origin main`

Important Beginner Tip: Empty Repository

✓ Recommended Option A

Start locally and create an **empty** repository on GitHub – without a README, .gitignore, or license.

✓ Recommended Option B

Create it on GitHub first (with a README, etc.) and then **clone** the repository – this way the history is correct from the start.

- ❏ If the remote repository already has an initial commit and your local repository also has its own commits, the histories will diverge. The first push will then fail – this is a **Git protection feature**, not a GitHub error.



CLONING

Cloning an existing repository

Cloning means: You pull an existing repository from GitHub onto your computer —complete with the full Git history. This is not to be confused with a ZIP download, which does not contain a Git connection.

Cloning: When and how

When to clone?

- The repository already exists on GitHub
- You want to work on a third-party or existing project
- You need the complete Git history locally
- You don't want to manually create an empty directory

What is automatically available after cloning

- Local project folder
- Complete commit history
- Default remote `origin` (automatically set up)
- Checked-out default branch

Command: `git clone https://github.com/YOUR-NAME/my-project.git` – or via SSH: `git clone git@github.com:YOUR-NAME/my-project.git`

SYNCHRONIZATION

Synchronization with GitHub

Synchronization means that your local state and the state on GitHub are aligned with each other. In Git, you primarily synchronize **commits and branch states** – not just loose files.

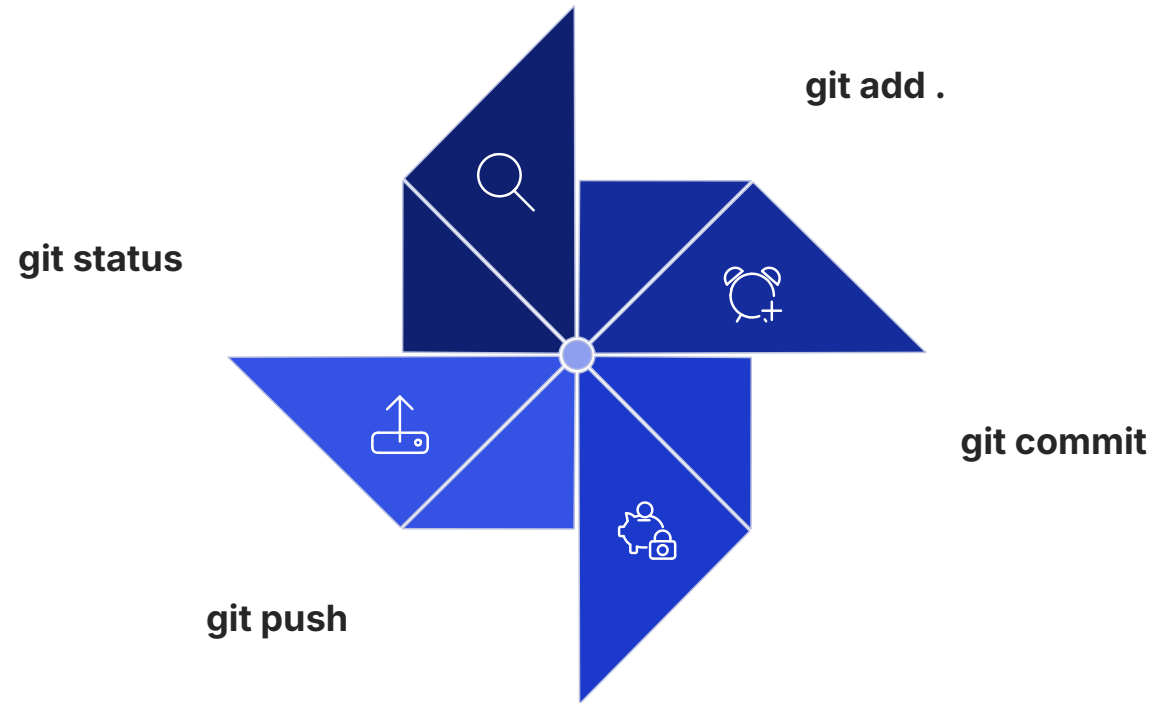
↑ Local → GitHub

You push your changes: `git push`

↓ GitHub → Local

You fetch or pull changes: `git fetch` or `git pull`

Uploading changes: The standard workflow



❏ **Common misconception:** `git add` and `git commit` do not upload anything to GitHub yet. Only `git push` transfers commits to the remote. This distinction is central.

When git push doesn't work

Common error messages and their meanings:

rejected / non-fast-forward

There are already changes on GitHub that you don't have locally yet. First, pull the remote state and integrate it.

authentication failed

You are not authenticated correctly. Password authentication is no longer possible – a Personal Access Token or SSH is required.

No upstream set

When pushing a new branch for the first time: `git push -u origin feature-login` sets the linkage.

fetch vs. pull – a key difference

git fetch

Downloads new commits from the remote, but **does not automatically** integrate them into your branch. The clean, cautious approach – you can first check what has changed.

```
git fetch origin
```

git pull

Combination of fetch and merge: Downloads changes and **integrates them immediately**. Practical, but sometimes too automatic for learning purposes.

```
git pull origin main
```

Recommended safe workflow: `git fetch origin` → `git status` → `git pull origin main`

COLLABORATION

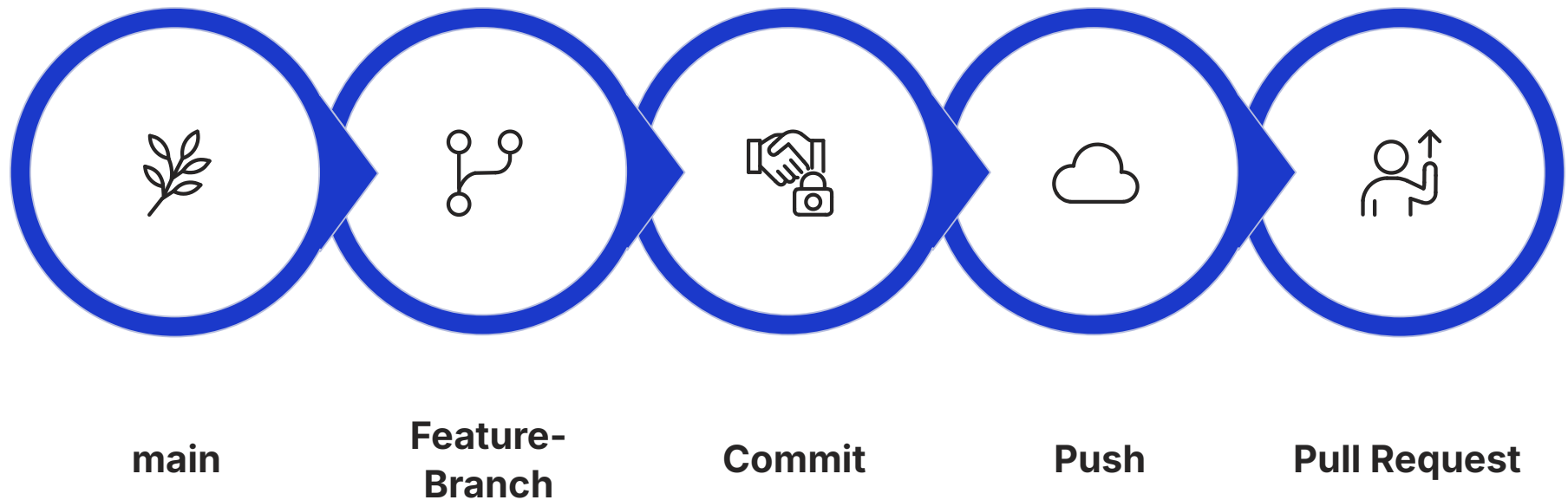
Collaboration with GitHub

GitHub is not just a repository for code – the platform is built primarily for collaboration. Anyone who understands branches, pull requests, and merge processes understands the true value of GitHub.



Pull Requests: Core Principle

A **Pull Request** is the standard GitHub way to propose changes from one branch into another – for review, discussion, and subsequent merge.



A Pull Request separates **writing code** from **releasing code** – this makes changes verifiable and keeps the main branch stable.

What you see in a Pull Request



File Changes

Which files and lines were added or removed—clearly highlighted.



Reviews & Comments

Who commented on what, whether checks were successful, and if there are any conflicts.



Merge Status

Whether the branch can be merged without conflicts and which branch is the target (**Base** vs. **Compare** branch).

❏ Even if you are working alone, pull requests are useful—they force you to treat your change as a self-contained, verifiable unit.

Identifying and Resolving Merge Conflicts

A **merge conflict** occurs when the same lines in competing branches have been edited differently. Git marks the conflict in the file:

```
<<<<<< HEAD
Hallo Welt
=====
Hallo GitHub
>>>>>> feature-begruessung
```

What the markers mean

```
<<<<<< HEAD = Version of your current branch
===== = Separator line
>>>>>> = Version from the other branch
```

Solution in 3 steps

1. Edit file manually – keep the desired content
2. Remove all conflict markers and save
3. `git add` → `git commit`

Avoiding conflicts: Good practice

The longer a branch remains separated from the main branch, the greater the risk of conflicts.

→ **Make smaller changes**

Manageable, focused commits
reduce overlaps with other branches.

→ **Synchronize frequently**

Regularly run `git pull` or `git fetch` to
keep the local state up to date.

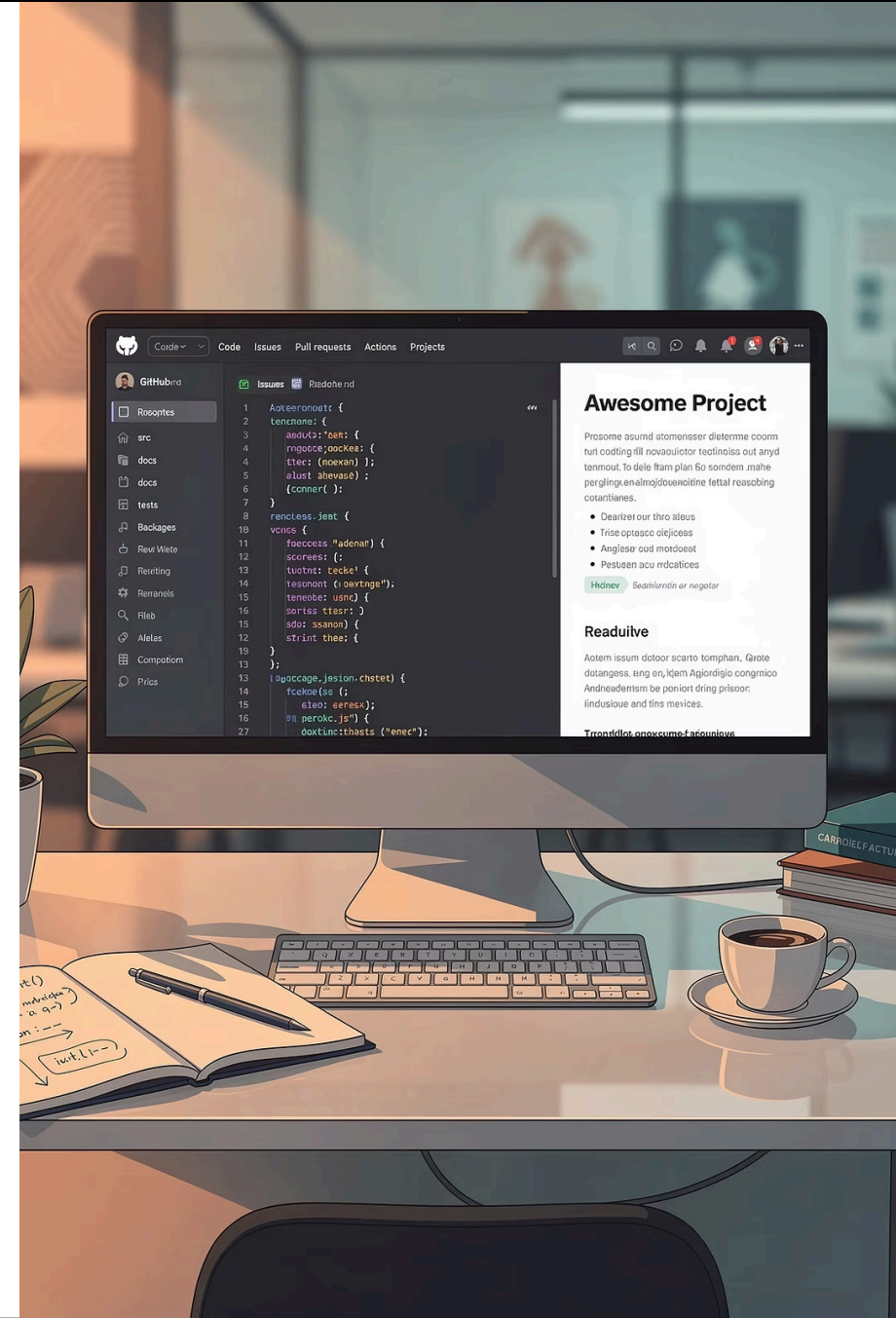
→ **Don't leave Pull Requests
open too long**

Long-lived branches diverge more –
early merging significantly reduces
conflicts.

DOCUMENTATION

Documentation on GitHub

A good repository consists of more than just working code. Documentation is not "extra work," but part of good software development—it helps you just as much as it helps others.



Navigating the GitHub Web Interface

When you open a repository on GitHub, you will find several key areas:

Code	Files, folders, branch selection, cloning, commits
Issues	Tasks, bug reports, discussions
Pull requests	Change proposals and reviews
Actions	Automations and CI/CD workflows
Projects	Project organization
Wiki	Additional documentation
Insights	Statistics and activity data

❏ Those who only see GitHub as a "place to upload" are perhaps only using 20% of the platform. Code, Issues, and Pull Requests are particularly central.

Issues vs. Pull Requests

Issue

A problem, a task, or an idea. No code – rather a discussion or a request. Managed under the **Issues** tab.

Pull Request

A concrete code change as a proposal. Contains commits, file comparisons, and review comments. Managed under **Pull requests**.

A common beginner mistake is to confuse issues with pull requests. Both are important tools – but for different purposes.

README.md: The Gateway Documentation

The **README.md** file is the standard introductory documentation for a repository. GitHub displays it prominently on the main page. **README** means "read me first," and **.md** stands for Markdown.

What is the project?

Clearly state the name, description, and purpose of the project.

How do you start it?

Installation and startup instructions so others can get the project running locally.

How do you use it?

Key usage steps, technologies used, and the current project status.

Essential Markdown Elements

Markdown is intentionally kept simple. A few elements go a long way:

# Heading	Large heading (H1)
## Subheading	Smaller heading (H2)
- Item	Bullet point
`code`	Inline code
```bash ... ```	Code block
[Text](URL)	Link

- ❑ **Learning effect:** Once you can document something clearly, you usually have a much better understanding of it. A good README is especially valuable for learning projects.

# Best Order for Documentation

This is how your repository on GitHub becomes not only technically correct, but also truly readable and usable for others.



---

**Project Name & Short Description**

---

**Purpose of the Project**

---

**Getting Started & Installation  
Instructions**

---

**Key Usage Steps**

---

**Project Structure or Special Features**

# Summary: Key Concepts

1

## Local vs. Remote

Cleanly separate the three states: files, local repo, GitHub.

2

## Connecting & Cloning

`git remote add origin` or `git clone` – depending on the starting situation.

3

## Push & Pull

`add` → `commit` → `push`. Fetch before pull for more control.

4

## Pull Requests & Conflicts

Separate writing code from sharing code. Recognize and resolve conflicts early.

5

## README.md

Documentation is part of good development – clear, useful, up-to-date.